

Differentiable visual computing for inverse problems and machine learning

Received: 8 March 2023

Accepted: 19 September 2023

Published online: 17 November 2023

 Check for updates

Andrew Spielberg^{1,2} , Fangcheng Zhong³ , Konstantinos Rematas⁴,
Krishna Murthy Jatavallabhula², Cengiz Oztireli^{3,4}, Tzu-Mao Li⁵  &
Derek Nowrouzezahrai⁶ 

Modern 3D computer graphics technologies are able to reproduce the dynamics and appearance of real-world environments and phenomena, building on theoretical models in applied mathematics, statistics and physics. These methods are applied in architectural design and visualization, biological imaging and visual effects. Differentiable methods, instead, aim to determine how graphics outputs (that is, the real-world dynamics or appearance) change when the environment changes. We survey this growing body of work and propose a holistic and unified differentiable visual computing pipeline. Differentiable visual computing can be leveraged to efficiently solve otherwise intractable problems in physical inference, optimal control, object detection and scene understanding, computational design, manufacturing, autonomous vehicles and robotics. Any application that can benefit from an understanding of the underlying dynamics of the real world stands to benefit substantially from a differentiable graphics treatment. We draw parallels between the well-established computer graphics pipeline and a unified differentiable graphics pipeline, targeting consumers, practitioners and researchers. The breadth of fields that these pipelines draw upon—and are of interest to—includes the physical sciences, data sciences, vision and graphics, machine learning, and adjacent mathematical and computing communities.

Originally designed for applications in computer graphics, visual computing (VC) methods synthesize information about physical and virtual worlds, using prescribed algorithms optimized for spatial computing. VC is used to analyse geometry, physically simulate solids, fluids and other media, and render the world via optical techniques. These fine-tuned computations that operate explicitly on a given input solve so-called forward problems, which VC excels at. By contrast, deep learning (DL) allows the construction of general algorithmic models, side-stepping the need for a purely first-principles-based approach to problem solving. DL is powered by highly parameterized neural network

architectures—universal function approximators—and gradient-based search algorithms, which can efficiently search that large parameter space for optimal models. This approach is predicated by neural network differentiability, the requirement that analytic derivatives of a given problem's task metric can be computed with respect to the neural network's parameters. Neural networks excel when an explicit model is not known, and neural network training solves an inverse problem in which a model is computed from data.

Although VC provides a strong inductive bias about the dynamics of real-world phenomena—one that would otherwise have to be learned

¹School of Engineering and Applied Sciences, Harvard University, Cambridge, MA, USA. ²Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge, MA, USA. ³Computer Science and Technology, University of Cambridge, Cambridge, UK. ⁴Google, Mountain View, CA, USA. ⁵Computer Science & Engineering, University of California — San Diego, San Diego, CA, USA. ⁶Electrical & Computer Engineering, McGill University, Montréal, Québec, Canada. ✉e-mail: aespielberg@seas.harvard.edu; derek@cim.mcgill.ca

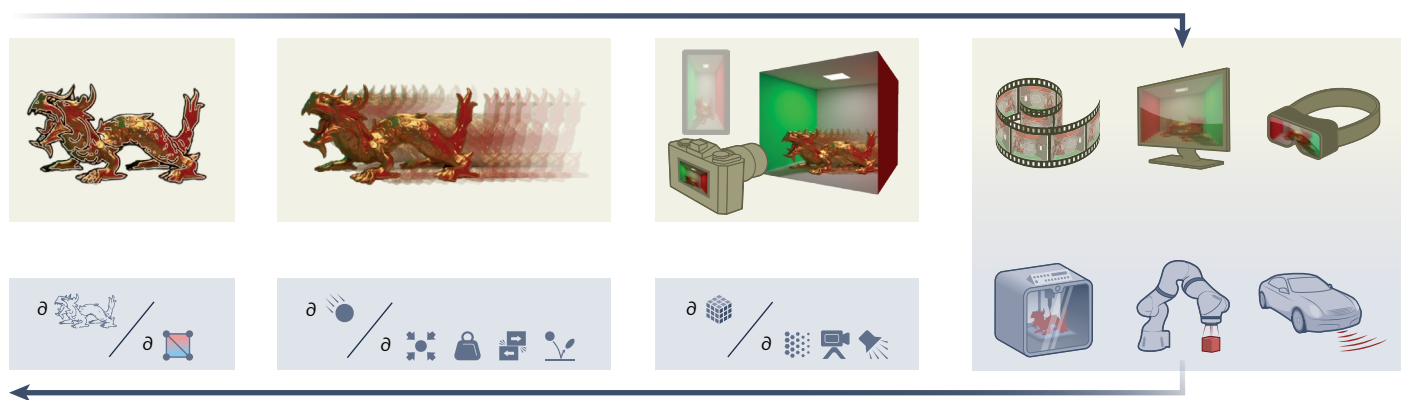


Fig. 1 | Forward and backward VC pipeline diagram. Top: VC steps through (left to right) the geometric representation, dynamical evolution and image synthesis of virtual 3D scenes, resulting in (far right) output image sequences (top row) or inferences for applications such as digital manufacturing, industrial robotics and autonomous driving (bottom row). This forward pipeline can be used, for example, for applications in visual effects, computer-aided design, virtual

reality and medical imaging, to name a few. Bottom: the DVC pipeline aims to compute the sensitivities of associated forward pipeline components (geometry processing, animation and rendering) with respect to variations in their inputs. Most immediately, such a pipeline can be aimed at a diversity of inverse problems, where knowledge of underlying physical processes can be imbued into task-based learning objectives.

from scratch in a pure DL context—its inability to adapt its mathematical models based on observations of real-world phenomena precludes its direct integration into larger DL-based systems. Meanwhile, DL is adaptive but requires a sufficiently expressive neural network, high-quality and diverse data observations of the underlying dynamics, and time to train the model to formulate (potentially complex) relationships between observations and dynamics. Differentiable visual computing (DVC) is a nascent family of approaches that aims to bridge this divide.

DVC combines the accurate, compact and explicit models of reality with system parameterization and gradients needed to adapt their parameters, that is, using gradient-based optimization. DVC constrains otherwise general, but data-inefficient DL methods to the boundaries of physical reality. Thus, the combination of DL- and DVC-based methods can significantly improve the data efficiency, speed and accuracy with which machine learning can be applied to inference problems in real-world physical systems. Despite its infancy, DVC is unlocking applications in areas such as physical reasoning from video, biomechanics modelling, robot control, automated design for manufacturing, and autonomous driving.

We present our vision of a complete DVC pipeline, shown in Fig. 1, that combines advances in differentiable geometry, animation and physics, not only to generate visual content, but also to efficiently understand and improve it using gradient-based information. We provide a holistic perspective on the spectrum of DVC problems, relating them to their analogues in the computer graphics domain while highlighting the additional expressive power they can leverage through their marriage with modern DL techniques. Perhaps most importantly, we discuss the potential of DVC to substantially reduce the amount of data modern DL methods require, while improving their generalization and interpretability.

We proceed by reviewing the preliminaries of differentiable programming. We then expand to the vision of the DVC pipeline and review state-of-the-art methods for its constituent parts, and finally conclude with the state of the field, including infrastructure, challenges and opportunities. Throughout, we draw parallels in challenges shared between fields and opportunities for the cross-pollination of approaches with DL.

Preliminaries on differentiable models, autodifferentiation and visual versus learned models

In the era of data abundance, the success of modern machine learning is predicated on two key technologies: expressive differentiable

computation models and sophisticated gradient descent-based optimization routines that specialize these models to target applications and tasks.

Concretely, a model $f: \mathbf{x} \in \mathbb{R}^n \rightarrow \mathbf{y} \in \mathbb{R}^m$ parameterized by $\boldsymbol{\theta} \in \mathbb{R}^p$ is said to be differentiable if derivatives of any $y \in \mathbf{y}$ can be evaluated with respect to any $x \in \mathbf{x}$ and $\theta \in \boldsymbol{\theta}$. Note that f could be an analytical function or a complex algorithm of indeterminate length (such as an optimization procedure; Fig. 2). In practice, guarantees on gradient behaviour—for example, that they be nonzero (in most places), sufficiently smooth and efficient to compute—are necessary for performant gradient-driven optimization.

Perhaps the most well-known differentiable models today are neural networks, which are employed in DL (an extensive introduction to the topic is provided in ref. 1). Typically constructed as high-parameter compositions of differentiable primitives (layers), neural networks are employed—with great success—across a stunningly diverse set of tasks. Each network layer is designed such that the gradient of any output of the layer can be computed with respect to its inputs and its parameters. Using the chain rule, gradients can then be computed across the entire network. A typical workflow computes the gradient of a scalar output computation (often called a ‘loss’) with respect to the network parameters, conditioned on some input dataset. The loss specifies a task by minimizing a penalty function (for example, the error of a prediction $\mathbf{y}$ with respect to known ground truth $\hat{\mathbf{y}}$). Using these gradients, optimization procedures can efficiently search a high-dimensional parameter space to minimize a loss and thus maximize task performance.

As DL typically optimizes a single loss with respect to a very high-dimensional parameter set, backward-mode autodifferentiation (also called backpropagation) is employed. Backpropagation computes the gradients of a loss with respect to its inputs by recursively computing and composing gradients backwards through the network layers, eventually generating gradients of the loss with respect to every network parameter. Figure 2 presents an illustration. Computing the layer-wise Jacobian-vector products needed to compose gradients according to the chain rule requires knowing the values of the outputs of each layer. Thus, this ‘backward’ pass is preceded by a ‘forward’ evaluation of the network (and loss), during which layer evaluations are computed and cached. This procedure is memory-intensive for deep and/or high-dimensional models, but keeps the gradient computation runtime efficient.

The mathematical underpinnings of differentiable visual models mirror those of DL: we can decompose a DVC model into a set of

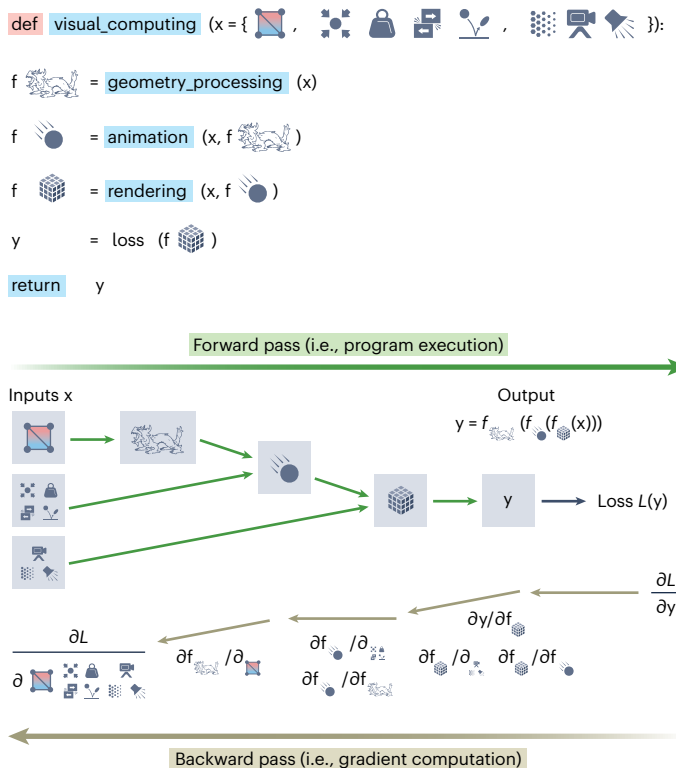


Fig. 2 | Forward and backward evaluation of a DVC program. Given a forward computational model (expressed mathematically or programmatically) such as the VC pipeline (left), we can compose the partial derivatives of the intermediate outputs of each stage of computation with respect to their inputs to arrive at the total derivative of the final output with respect to the original inputs (right). Automatic differentiation⁹⁹ and specialized methods can be used to transform forward models (such as the code block; left) into the backward model, which is useful for gradient-based optimization. Currently prevalent competing approaches replace the specialized VC processing blocks with more standard nonlinear computational blocks from neural network architectures, such as convolutional or fully connected operations.

subcomputations (cf. layers), whose relation is specified by a general directed acyclic computation graph (DAG, cf. neural network). We can similarly backpropagate through the visual model's DAG to compute gradients, assuming we first cache the outputs of each DAG node during forward evaluation steps. The building-block computation nodes (cf. neurons) in DVC models include geometry processing, physics-based animation and image synthesis (more commonly referred to as rendering; Fig. 1). Although most DVC models use backpropagation for differentiation, certain problems are created by algorithms that require gradients of many outputs of a model with respect to a single input/parameter, or the gradients of many outputs with respect to many inputs/parameters. In such cases, forward and mixed differentiation schemes can be preferable. We touch upon these alternative strategies throughout this survey.

The power of differentiable visual models is that they facilitate efficient gradient computation, which can substantially outperform finite differencing schemes in speed, accuracy or both, or the need to learn a differentiable model. This saves time and diminishes the need for data, which can be expensive to gather or generate. However, visual models are often more complex than neural networks; their gradients are more expensive to compute and their representations are even more nonlinear, which can lead to numerical issues and analytical search algorithms becoming trapped in local minima. The remainder of this Review takes a magnifying glass to how these visual models are constructed so as to increase their efficacy, as well as how they can be

composed with other visual and DL models in cutting-edge real-world applications.

The forward and backward VC pipelines

The classic computer graphics pipeline proceeds by instantiating geometry, animating it as desired, and then rendering the geometry for visual appeal. At a high level, this is not so different from the way scientists and engineers reason about nature and design—first defining a geometric representation for their problem, simulating its behaviour, and then analysing it based on (often visual) observation. In fact, modern simulation-powered computer-aided design (CAD) software presents an example of such techniques operating simultaneously in the contexts of scientific analysis, design and engineering, and computer graphics. In this section, we provide a high-level overview of the classical VC-pipeline forward process and our vision of its differentiable analogue.

DVC enables each computer graphics process to be differentiable, as well as the end-to-end pipeline. Imagine one wishes to design a scene for an animated fantasy film. We might first specify the geometry of a dragon, animate its motion for a scene, and then render the dragon to be viewed on film. As each section (geometry, animation, rendering) is a differentiable process, one could optimize the dragon's shape, control and lighting to make it look as bold, scary, bright and so on, as possible (Fig. 1) by computing gradients of that objective and using them for numerical optimization. Although end-to-end differentiable pipelines are not presently used in production, this vision is not a pipedream, as differentiable methods are increasingly employed in domain-specific industrial tasks (for example, constraint satisfaction in CAD, optimal control in animation, and accurate illumination in rendering).

At the research level, however, differentiable methods across domains are already being combined for diverse applications. For example, ref. 2 shows the co-optimization of the design and behaviour of simulated robotic fish, combining the use of differentiable geometry and animation. Reference 3 combines differentiable rendering and geometry techniques to estimate shape from images. Reference 4 demonstrates inference of the materiality of physical objects directly from rendered video, while ref. 5 applies a similar approach to estimate human motion. Recently, ref. 6 demonstrated the extraction of geometry, animation and rendering parameters from multiview video, effectively combining all elements of the differentiable pipeline. Each of these works also demonstrates how DVC layers can be seamlessly integrated with traditional DL models, which can be trained end-to-end or in their own standalone processes⁷. The integration of DL models makes it possible to learn otherwise difficult-to-model phenomena (including difficult-to-model physics or user-aesthetic preferences) and include these in optimization-driven computer graphics or science applications.

Unfortunately, the physical world is full of noise and discontinuities. Modelling physical phenomena in a way that is accurate and useful (that is, the correct prior), but also differentiable, remains a core salient challenge. We next describe differentiable techniques in geometry, animation and rendering, and describe the challenges and advances in each.

Geometry

The VC pipeline begins with geometry. For something to move or be seen, it must exist, and geometry specifies where 'things' exist and where they do not. These things could be tangible, like an apple falling due to gravity, or matterless but still physical, like an area light source, or completely abstract, like a region for numerical integration. Regardless, the definition of a geometric object is always mathematical, with direct or algorithmic means of specification.

The ability to use gradients to optimize geometry is directly powerful in engineering applications such as surface reconstruction⁸, topology optimization⁹, designing printed objects with desired mechanical

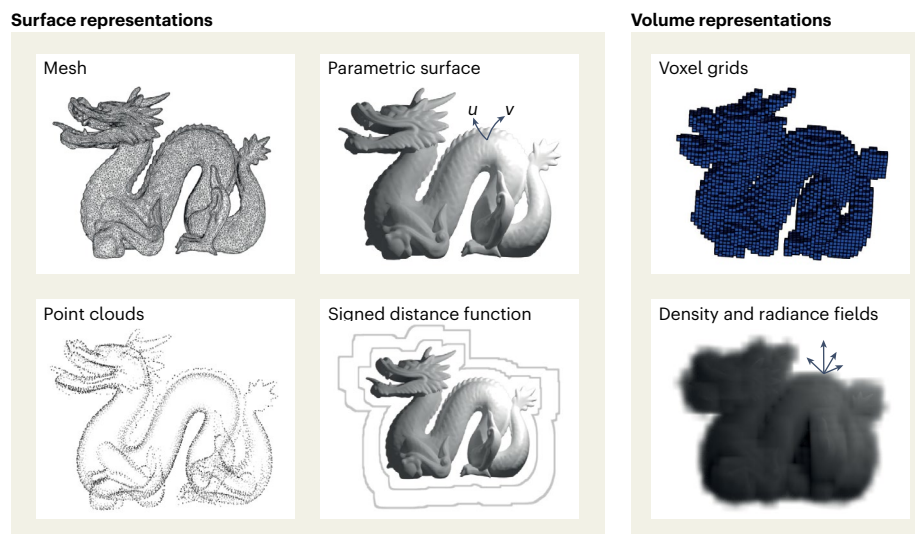


Fig. 3 | Visual illustration of common surface and volumetric geometric representations. Each representation has varying degrees of specification, simplicity and differentiability. The more discretized representations (meshes, point clouds, density and voxel grids) are typically more difficult to differentiate

through, but are simpler to express. Meanwhile, parametric surfaces, signed distance functions, and density and radiance fields are more difficult to compute, but typically emit more useful gradients.

properties^{10,11}, and robot morphology optimization². Recent experimental CAD kernels have been introduced that allow for differentiating through the geometry specification process^{12–14} and optimize for manufactured performance at design time.

Representations in geometry typically fall along three axes: (1) volumetric versus surface, (2) explicit versus implicit and (3) discrete versus continuous, as illustrated in Fig. 3. Volumetric representations specify an occupancy or density value for every position in a three-dimensional (3D) volume. Such specification can be either discrete (for example, voxel grids^{15,16}) or continuous (for example, occupancy/density field^{17,18}). Volumetric representations are the most general form for characterizing a 3D scene and are robust in modelling complex scene geometries and topologies such as hair, fabric and smoke. This generality comes at the expense of memory efficiency, as each segment of space must be explicitly specified. Furthermore, volumetric representations lack certain precomputed information useful for applications, such as surface normals. Surface representations, on the other hand, explicitly or implicitly specify boundaries of volumes. These representations are compact and often more geometrically expressive compared to volumetric representations, but can impel additional computations, such as checking if a point is interior. Explicit representations can be unions of discrete boundary elements (for example, meshes¹⁹) or continuous functions (for example, parametric surfaces²⁰, and splines²¹) while implicit approaches specify a surface by identifying the level set of a trivariate function (for example, a signed distance function). Although implicit representations are more general, explicit geometry allows direct computation of a value (such as distance from a boundary) at any point in space. Meanwhile, discrete representations sacrifice precision for simplicity. Because each representation has its own pros and cons, they co-exist in VC; some hybrid methods operate across multiple representations^{22,23}.

Representations have advantages and disadvantages in DVC as well, and geometric representations are further chosen according to the type of gradient information needed for the downstream task. In practice, geometries can be (differentiably) converted between representations within an algorithm, and different representations can be used for analysis (training) and deployment (inference). If a representation is or can be made continuous and smooth, the recipe for computing gradients is straightforward: first, perform the forward pass, and then, algorithmically perform the backward pass via

backpropagation (Fig. 4). The question then is how to emit continuous derivatives from discrete representations, especially since the most geometric representations are discrete (for example, bitmaps in 2D and triangle meshes in 3D). Three core methods exist for smoothly reasoning about discrete geometry: (1) smoothing the domain, (2) smoothing the output and (3) learning continuous representations.

Domain smoothing involves transforming a discrete variable into a continuous one, with the expectation that a downstream algorithm will force its value to a valid integer value. Examples include topology optimization, which smoothly interpolates voxels between vacancy (0) or occupancy (1), but forces them to 0 or 1 in a downstream algorithm⁹. (Neural approaches to such geometry design have also been proposed^{24,25}.)

Output smoothing occurs when consuming algorithms smooth the domain so as to interpolate the space between discrete regions such as vertices in a mesh. Classic examples include pixel filters (used in anti-aliasing) and finite element interpolation, which approximates values at nodes of volumetric meshes across their interior²⁶. Other more sophisticated methods include multi-resolution kernels²⁷ and neural methods such as diffusion models²⁸ and neural subdivisions²⁹.

One of the fastest growing areas in differentiable geometry processing comprises neural operators, which seek to replace parts of or entire algorithms with domain-agnostic differentiable models that can be learned offline or embedded in a DVC algorithm. These operators provide useful gradients and can still reason about discrete structures. One common approach is to embed discrete inputs as continuous tensors that encode the structure of a geometry. For example, for point clouds, PointNet has seen substantial adoption^{30,31} (more generally it operates on any unordered set). It has also been modified for implicit point selection³² and improved local geometric structures³³. Approaches have also been proposed for deep representations for reasoning about signed distance fields³⁴, manifold graph embeddings³⁵ or mesh collections³⁶. In other cases, neural operators exist that replace entire algorithms. PointFlow provides a multi-resolution method for converting meshes to point clouds³⁷. Reference³⁸ describes a method for mesh deformation using neural fields, and ref. 39 proposes a framework for learning generic geometric operators.

The process of deriving useful gradients in geometry processing shares many of its challenges with physics and rendering: namely, how to continuously reason about discontinuous structure (cf. contact in

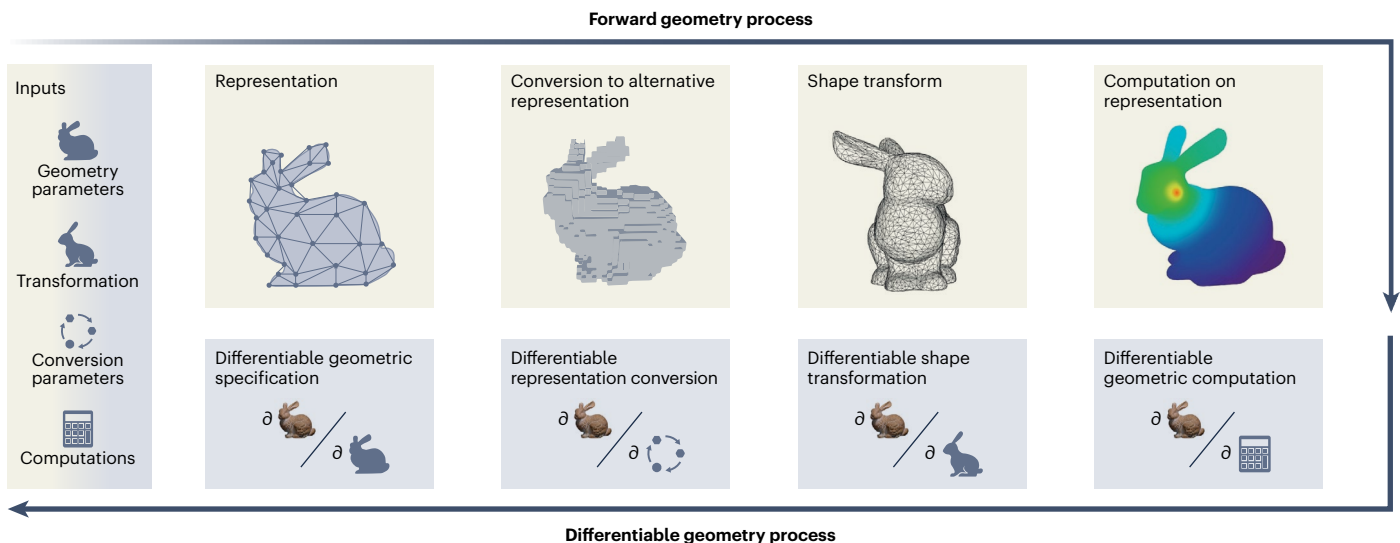


Fig. 4 | Forward and differentiable geometry processing. Geometry processing deals with problems surrounding (top row, left to right) the discrete representation, conversion between alternative representations, potential transformations, and computation of properties of the surfaces of geometric

objects. In the differentiable setting, gradients of (potentially continuous) geometric properties are sought with respect to (bottom row, right to left) measured signals on their manifolds, transformations applied to them, and the underlying discretization and representational parameterizations.

physics and occlusion in rendering), and how to most effectively combine analytical and learned models. In the next section, we describe how gradients are useful for reasoning about the motion and deformation of geometries specified by differentiable representations.

Animation

Physically based animation, also referred to as physics simulation, provides a means of computationally predicting, visualizing and analysing the motion and interaction of objects in the world. It is used to design and control industrial machines and robots, predict the behaviour of novel materials and provide high-quality computer-generated imagery for movies and real-time dynamics for digital games. In this Review we focus primarily on dynamic, time-varying simulators, although static simulation introduces its own interesting challenges for both forward and backward computation^{40,41}.

Formally, a physics simulation computes the time evolution of a system by computing the change of the system $\dot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, t)$ at time t , where state variable $\mathbf{q}$ could be compact (in the case of finite rigid degrees of freedom) or infinite in cardinality (in the case of continuum mechanics). This rate of change is then time-stepped forward via time integration. Because it is intractable to compute the state evolution at every point in time, physical systems are typically time-integrated in discrete steps dt . Coupling the rate of state change and time integration can be interpreted as a single state transition function $\mathcal{S}$, where $\mathbf{q}_{i+1} = \mathcal{S}(\mathbf{q}, t, dt; \boldsymbol{\theta})$. Here, $\boldsymbol{\theta}$ refers to system parameters, which could include a controller, environmental parameters, design parameters and so on.

The emerging differentiable physics simulators provide a means of both predicting reality and better understanding it. Although forward simulation allows one to, say, predict a robot's motion, gradients allow one to efficiently improve it. As another application, gradients can allow one to efficiently search for the assigned material parameters of a real-world elastomeric object, by searching for, for example, stiffness or damping coefficients such that a simulated trajectory matches a real-world measured trajectory. Compared with corresponding gradient-free methods such as reinforcement learning or evolutionary strategies, gradient-based methods can be orders of magnitude more efficient^{42–46}.

Two prominent strategies exist for constructing differentiable physical simulators—‘explicit’ methods and ‘learning-based’ methods.

Explicit methods derive simulators—and their gradients—from first principles, programming a model of the time evolution of a physical system and algorithmically deriving its gradients. By contrast, learning-based methods focus on learning a state-transition model from observed simulated or real-world data. These methods require less domain knowledge but typically rely on physics-inspired neural network architectures that are catered to the structure of the state evolution^{47,48} or matter arrangement^{49,50}. In either case, a complete dynamic simulation is constructed by recursively applying the state transition model for the desired duration of the simulation.

Methods for making simulations differentiable are agnostic to the state-transition model, as long as the model itself is differentiable. A common strategy is as follows: given a differentiable state transition model $\mathcal{S}$, one ‘rolls-out’ a system trajectory by iteratively applying $\mathcal{S}$, such that the final state is $\mathbf{q}_T = \mathcal{S}(\dots \mathcal{S}(\mathbf{q}_0) \dots)$ ^{42,46,51}. The system state at each intermediate state of the simulation is cached along the way before backpropagating gradients of a loss $L(\mathbf{q}_T)$ back to the beginning of simulation, with system gradients with respect to states and parameters $\frac{\partial L(\mathbf{q}_T)}{\partial \boldsymbol{\theta}}, \frac{\partial L(\mathbf{q}_T)}{\partial \mathbf{q}_{T-1}}$ (Fig. 5). This technique has been applied to a large

number of domains, including classical (contact-rich) Newtonian mechanics and rigid robotics^{46,52,53}, elastic continuum mechanics and soft robotics^{42–44,49,54}, cloth physics⁵⁵ and fluid dynamics^{56,57}, and has been applied to problems such as system identification, robot control learning and optimization, computational design and dynamical extrapolation. A summary of some seminal work, as well as their contributions, are described in Fig. 6.

The long time-evolution of physical simulation distinguishes it from geometry and other domains of VC and introduces key challenges. Vanishing gradients, which also arise when differentiating deep neural networks, can numerically diminish the effect of one variable on another. Shortening the gradient horizon and handling the simulation in sub-sequences is one strategy for overcoming this issue⁵². Another problem is that of memory consumption, as long simulations require many intermediate states to be stored in memory. This issue can be partially ameliorated by domain-optimal checkpointing strategies⁵⁴ or decreased precision⁵⁸.

In the case of long simulations where one cares about only a few system variables (and memory is precious), ‘forward-mode’ differentiation may be employed. As states evolve forward, a Jacobian relating the

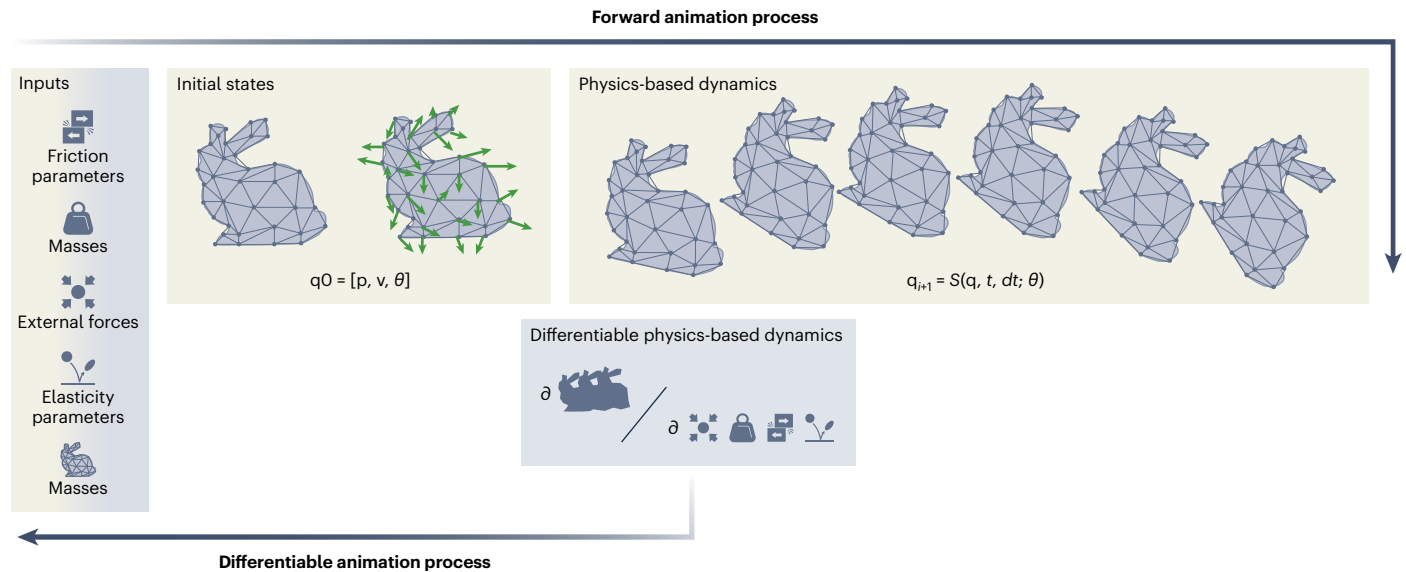


Fig. 5 | Forward and differentiable animation. In the forward simulation (top), scene dynamics parameters (far left box) are used in a numerical simulation of the time evolution of the dynamics of motion (top row, left to right). Conversely, in the differentiable animation setting (bottom), we aim to compute the

gradients of the simulation outputs (that is, body locations over time, spatial-temporal collision events and so on) with respect to the dynamics parameters (bottom middle).



Fig. 6 | Differentiable animation applies to a diversity of problems where system identification and unknown dynamics parameters need be inferred from (3D/4D) observations. Application domains include robotics, policy

learning, computational manufacturing and scene understanding. Panel references (top to bottom): left column, refs. 42–44; middle column, refs. 51,52,61; right column, refs. 49,56,57.

gradients of output states to q_0 and θ is maintained and accumulated forward in time, again employing the chain rule. This occurs ‘on the fly’; gradients are computed during the forward simulation pass, never requiring system states to be cached in memory. Jacobian computation has historically been a workhorse in optimal control (linear quadratic regulators, trajectory optimization^{59,60}), and many robotics toolboxes compute such gradients efficiently^{61,62}. Because forward-mode differentiation can only differentiate with respect to a single variable at a time (cf. backpropagation and a single loss function), forward-mode differentiation is ill-suited to high-dimensional input/parameter problems.

Problem structures that are common in animation and control that would be slow or difficult to differentiate have been a major recent focus. In applications where state evolution relies on an optimization-based controller, or in quasistatic simulation, the loss

is the solution of an expensive optimization problem. In such cases, the implicit function theorem⁶³ or bespoke optimization ‘layers’⁶⁴ can perform efficient differentiation. Moreover, similar to differentiating through discrete structures in geometry, most simulation domains involve contact or other hard boundary conditions; these hard constraints can be repeated many times throughout a simulation. Choosing a smoothed, relaxed contact model or learning such a model⁶⁵ can alleviate gradient non-smoothness.

The frontier of differentiable physics is rich with challenges and opportunities. As dynamics can be particularly chaotic, gradient-based optimization is especially prone to suboptimal local minima. Algorithms that combine exploration (such as those in reinforcement learning or Monte Carlo tree search) with gradient-based optimization are one exciting horizon⁶⁶. More rigorous methods that combine the best

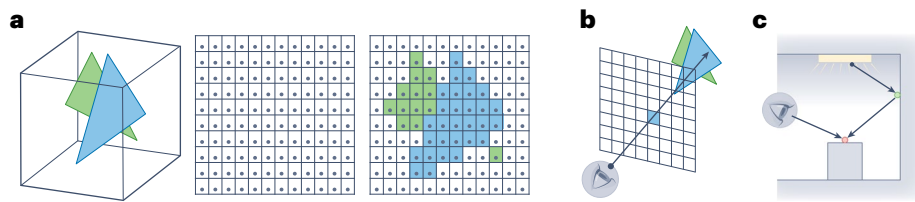


Fig. 7 | Visualization of rasterization, ray-tracing, and path tracing algorithms. **a**, Projection of triangles in 3D space (left) onto a 2D image plane (centre), followed by pixel-wise discretization (right). **b**, Ray-tracing images 3D

objects by determining where a camera ray would intersect with an object and appropriately colouring the image plane. **c**, Path tracing is used to realistically model the effects of illumination based on optical principles.

of ground-truth and learned models, such as residual neural networks⁶⁷ or adaptation layers⁶⁸, and analyses of the quality and usefulness of learned gradients^{69–71} each provide a path for creating robust models for real-world applications. Finally, we note that current physical simulators are usually domain-specific—differentiable multiphysics engines that operate across different time and energy scales are core to modelling the dynamical richness of reality.

Rendering

Rendering is the process by which geometry and action are seen. By understanding how (virtual) light reacts with the (virtual) world, the (virtual) world can be observed. By manipulating light, either through digital displays or physical pigments, the virtual world is then made physical, either through images or animations.

More formally, given as input a 3D representation of the world and an assignment of the material properties of objects (for example, plastic or wood) and the position of light sources and a virtual camera, rendering is the process of converting this 3D representation to a 2D image or 2D images. Rendering can encompass a broad range of such processes, spanning those that generate diagrammatic, illustrative or stylized image representations of the world to those that fully simulate the physical interactions of light and thus yield photographic-quality images. The latter is referred to as physically based rendering (PBR), and for a comprehensive treatment of this, see ref. 72.

VC relies primarily on two algorithmic paradigms when designing rendering algorithms: rasterization and ray-tracing. Rasterization-based frameworks operate by first performing a sequence of algebraic transformations to warp and project 3D scene elements onto a 2D imaging plane (Fig. 7a) before converting the now-2D vector form of the scene elements to a discretized representation on an image buffer. Rendering systems based exclusively on rasterization are favoured in high-performance applications, such as video games, as they are more easily parallelizable on GPUs. Traded off with this performance is flexibility: implementing PBR numerical methods on top of rasterization is markedly harder than using ray-tracing—and, in some case, is impossible. Ray-tracing, as its name implies, instead obtains a 2D representation of the 3D world by conceptually tracing rays of light in reverse—from the (virtual) camera, through each pixel, and into the 3D world—to query which 3D object it first interacts with (Figs. 7b and 1). Ray-tracing is better suited to PBR simulations of the underlying physical dynamics of light (called light transport), as it can be used not only to trace rays from the camera, but also from light sources and other objects. These simulations help determine which objects are visible to a viewer and how objects are lit by lights and scene reflections. Indeed, ray-tracing is extended in exactly this manner when simulating the emission, scattering and absorption of light in a 3D environment.

The simplest algorithmic realization of this process, path tracing, traces sequences of connected rays, called paths, from the camera and/or light source. This process is repeated with stochastic perturbation until an image is synthesized by collecting all light paths that connect a pixel on the camera sensor to an emitter; here, perturbations simulate

the various scattering interactions that occur at surfaces in the scene (Figs. 7c and 1). This process is useful not only for studying visible light, but any type of electromagnetic radiation, due to its ray-based nature⁷³.

Although the forward rendering process takes a fully defined 3D scene description and generates an image, the inverse rendering process infers information about the 3D scene given only image data as input. Many specialized realizations of inverse imaging have been explored in the foundational literature of computer vision, medical imaging, tomography and even computational astronomy. Most recently, differentiable rendering has grown to complement—and in some cases, subsume—these more specialized, domain-driven inverse approaches.

Concretely, so-called differentiable rendering approaches solve a variety of sub-problems in inverse inference, namely the numerical estimation of the gradient of a synthesized image with respect to (Fig. 8, left to right) geometrical changes such as occlusion, shading changes due to variability in material reflectance properties, lighting changes due to the multiple scattering of light energy in an environment, and viewpoint changes due to the many possible placements of a (virtual) camera sensor. These events are typically relatively smooth and continuously differentiable except at sparse event points.

The volume of scientific work addressing both end-to-end inverse rendering and its constitutive differentiable rendering sub-problems has grown rapidly in the last handful of years. The evolution of these methods follows similar historical trends as those in the forward rendering problem: the development of progressively more refined theoretical models of differentiable rendering^{74–77}, paradigm exploration surrounding the variants of rasterization^{3,78} and ray-tracing⁷⁴, which now include differentiable ones, and systems-oriented work to expose these tools to the broader machine learning community^{79–83}.

The growing ubiquity of neural scene representations, such as Neural Radiance And Density Fields (commonly shorthand as NeRF)¹⁸, has blurred the lines between differentiable rendering and neural-based approaches to the inverse problem. Here, differentiable rendering formulations custom-suited to an underlying (often neural) parameterization of the 3D scene have proven both effective (and necessary) in tackling the ill-posedness of the inverse scene estimation problem. Image-only supervision, particularly in the low-data regime, necessitates strong priors, that is, surrounding the image formation process. For an extensive summary of recent work on neural rendering, see ref. 84. Other neural scene representations have also been proposed, including methods with higher physical fidelity, but not necessarily leading to improved performance in downstream applications^{85,86}. When and why certain approaches outperform others remains an open question.

Open research challenges in differentiable rendering can be roughly categorized along numerical methods and algorithmic axes.

The development of more efficient numerical gradient estimators for the various differentiable rendering sub-problems remains an important area of exploration, especially in the context of their applicability to gradient-based optimization. For example, although numerical estimation in the forward rendering context focuses on

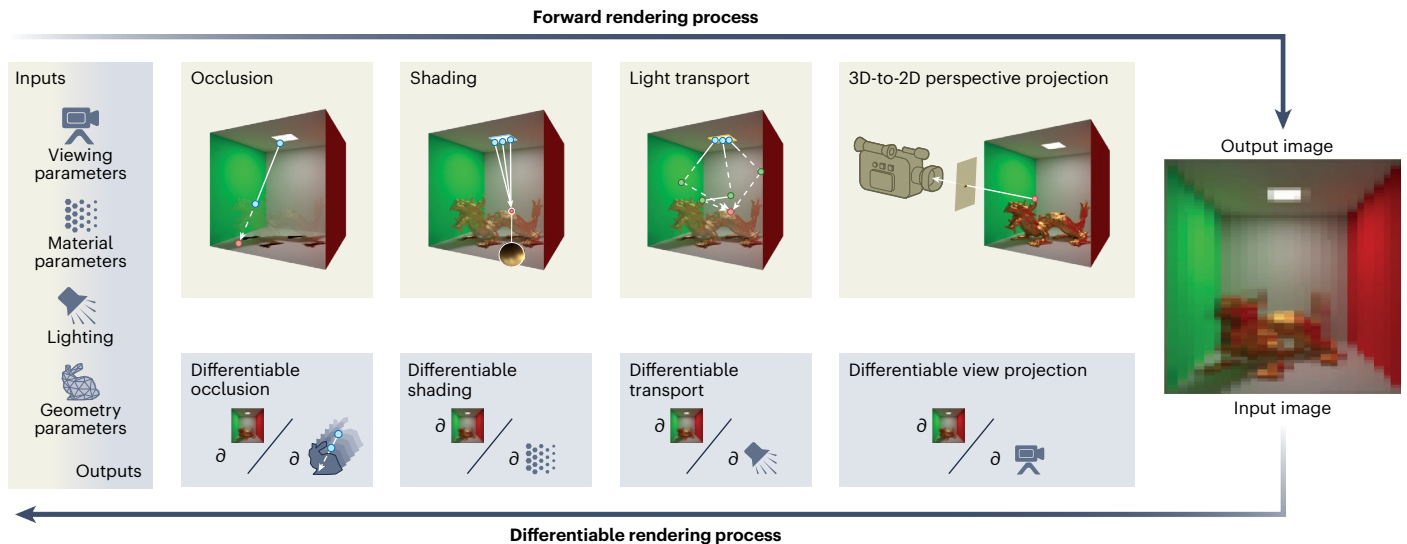


Fig. 8 | Forward and differentiable rendering. When synthesizing an image in forward rendering (top), scene parameters (far left box) are used to synthesize a realistic image (far right box), with (middle boxes, left to right) the geometric properties informing occlusion effects, material properties informing reflection and shading, lighting profiles informing complex multi-scattering effects, and viewing parameters defining the imaging sensor's response. The differentiable rendering context (bottom) focuses instead on the numerical estimation of

gradients of the synthesized image pixel intensities with respect to the 3D scene parameters, and can be similarly decomposed into the computation of image gradients with respect to (middle boxes, right to left) changes in the camera sensor's viewing parameters, complex light transport dynamics, localized shading and appearance routines, and occlusion due to potential variations in the scene geometry.

reducing image synthesis error, the focus of differentiable rendering algorithm development is notably different: variance in the gradient estimates does not necessarily invalidate their utility in gradient-based optimization. In fact, borrowing from many theoretical and practical explorations in the machine learning optimization community⁷⁰, gradient-based optimization methods may actually benefit from variance in minibatched gradient estimates, as these estimates remain useful when exploring loss landscapes in search of optima that admit lower generalization error. Algorithmically, from a computational complexity perspective, new techniques are emerging to address the tradeoff between runtime and the memory footprint in high-dimensional applications^{87,88}.

Discussion

The previous sections have presented a vision for a complete DVC pipeline beginning with the instantiation of scene geometry, continuing with animation via physically based or inspired techniques, through to rendering of the simulation to be viewed by humans or computers. The full differentiability of the pipeline means that gradients of any measurable quantity of the pipeline can be computed with respect to any variable in the system. These gradients allow for hyper-efficient optimization and learning algorithms that can out-class traditional methods and unlock new applications in science and engineering. For example, such a pipeline could enable effectual vision systems for autonomous driving, or allow robots to understand the physics of the world from vision and update their controllers in real-time changing environs. Furthermore, by increasing the efficiency of optimization and learning tasks, learned models are no longer the application, but are rather a tool to be used by even more complex algorithms. If such algorithms could, for example, allow a robot to quickly respond to human interaction, then robots could efficiently plan through predicted interactions, making them more versatile and robust.

This Review has focused on the three traditional pillars of computer graphics: geometry, animation and rendering. However, there remain topics in VC that are less explored and not covered in this

survey. Topics such as texture synthesis⁸⁹, performance capture⁹⁰ and procedural modelling⁹¹ are just a few more specific topics that, in recent years, have explored differentiable methods for improving performance and capability. Beyond topics that overlap with computer graphics, differentiable methods developed in VC have the potential to permeate to other aspects of science, such as biology⁹² or molecular dynamics⁹³.

Despite the promising results presented in this work, DVC as a discipline is in its infancy. Further algorithmic and systems-level advances will increase the efficiency, usability, applicability and, ultimately, ubiquity of differentiable methods. Rather than rely on the 'one simple trick' of backpropagation, new algorithms and systems will make differentiation more efficient^{94,95} and lead to the incorporation of other algorithmic methods from analysis (refs. 77,96). Domain-specific hardware and domain-specific compilers and languages will make it easier to implement new, computationally efficient models, and accelerate research and development times (our most precious resource). Indeed, such patterns have already begun to emerge with Halide^{97,98}, which supports chips developed specifically for hardware acceleration on smartphones, Dr. JIT⁸³, a differentiable framework for large computation graphs like those seen in VC, and Taichi^{58,79,80}, a GPU-accelerated differentiable sparse VC language. SDKs (such as TensorFlow graphics⁸² and PyTorch3D⁸¹), which provide quick-start access to the most popular differentiable methods, will allow researchers and practitioners to focus on complex problems rather than rote implementation, making DVC models as simple to deploy and integrate as neural networks are today.

DVC is a young field based on an old idea. In a data-driven world, differentiable methods dare to provide more and ask for less. They allow humans to insert centuries of scientific knowledge (that is, human learning) into the DL pipeline, making models more efficient to learn and more easily interpretable. Throughout the next decade, we expect to see a rich investigation of how to combine neural networks and VC models to extract the best of both approaches, and usher in a new era of machine intelligence that is based on both theory and empiricism.

References

- Goodfellow, I., Bengio, Y. & Courville, A. *Deep Learning* (MIT Press, 2016).
- Ma, P. et al. DiffAqua: a differentiable computational design pipeline for soft underwater swimmers with shape interpolation. *ACM Trans. Graph.* **40**, 132 (2021).
- Liu, S., Li, T., Chen, W. & Li, H. Soft Rasterizer: a differentiable renderer for image-based 3D reasoning. In *Proc. 2019 IEEE/CVF International Conference on Computer Vision* 7708–7717 (IEEE, 2019).
- Murthy, J. K. et al. gradSim: differentiable simulation for system identification and visuomotor control. In *International Conference on Learning Representations* (2020).
- Rempe, D. et al. Contact and human dynamics from monocular video. In *Proc. European Conference on Computer Vision (ECCV)* (eds Vedaldi, A. et al.) 71–87 (Springer, 2020).
- Peng, S. et al. Animatable neural radiance fields for modeling dynamic human bodies. In *Proc. 2021 IEEE/CVF International Conference on Computer Vision (ICCV)* 14294–14303 (IEEE, 2021).
- Spielberg, A. et al. Learning-in-the-loop optimization: end-to-end control and co-design of soft robots through learned deep latent representations. In *Proc. Advances in Neural Information Processing Systems* Vol. 32 (eds Wallach, H. et al.) 8284–8294 (Curran Associates, 2019).
- Kazhdan, M., Bolitho, M. & Hoppe, H. Poisson surface reconstruction. In *Proc. Fourth Eurographics Symposium on Geometry Processing* Vol. 7 (eds Fellner, D. W. et al.) 61–70 (ACM, 2006).
- Sigmund, O. & Maute, K. Topology optimization approaches. *Struct. Multidiscipl. Optim.* **48**, 1031–1055 (2013).
- Prévost, R., Whiting, E., Lefebvre, S. & Sorkine-Hornung, O. Make it stand: balancing shapes for 3D fabrication. *ACM Trans. Graph.* **32**, 81 (2013).
- Bächer, M., Whiting, E., Bickel, B. & Sorkine-Hornung, O. Spin-it: optimizing moment of inertia for spinnable objects. *ACM Trans. Graph.* **33**, 96 (2014).
- Banović, M. et al. Algorithmic differentiation of the Open CASCADE technology CAD kernel and its coupling with an adjoint CFD solver. *Optim. Methods Softw.* **33**, 813–828 (2018).
- Keeter, M. J. Massively parallel rendering of complex closed-form implicit surfaces. *ACM Trans. Graph.* **39**, 141 (2020).
- Cascaval, D. et al. Differentiable 3D CAD programs for bidirectional editing. *Comput. Graph. Forum* **41**, 309–323 (2022).
- Liu, L., Gu, J., Zaw Lin, K., Chua, T.-S. & Theobalt, C. Neural sparse voxel fields. *Adv. Neural Inf. Process. Syst.* **33**, 15651–15663 (2020).
- Fridovich-Keil, S. et al. Plenoxels: radiance fields without neural networks. In *Proc. 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 5491–5500 (IEEE Computer Society, 2022).
- Lombardi, S. et al. Neural volumes: learning dynamic renderable volumes from images. *ACM Trans. Graph.* **38**, 65 (2019).
- Mildenhall, B. et al. NeRF: representing scenes as neural radiance fields for view synthesis. *Commun. ACM* **65**, 99–106 (2022).
- Nicolet, B., Jacobson, A. & Jakob, W. Large steps in inverse rendering of geometry. *ACM Trans. Graph.* **40**, 248 (2021).
- Groueix, T., Fisher, M., Kim, V. G., Russell, B. & Aubry, M. AtlasNet: a papier-mâché approach to learning 3D surface generation. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (IEEE, 2018).
- Cho, M. et al. Differentiable spline approximations. *Adv. Neural Inf. Process. Syst.* **34**, 20270–20282 (2021).
- Chen, A., Xu, Z., Geiger, A., Yu, J. & Su, H. TensorRF: tensorial radiance fields. In *Proc. 17th European Conference on Computer Vision (ECCV 2022): Part XXXII* (eds Avidan, S. et al.) 333–350 (Springer, 2022).
- Müller, T., Evans, A., Schied, C. & Keller, A. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.* **41**, 102 (2022).
- Sosnovik, I. & Oseledets, I. Neural networks for topology optimization. *Russ. J. Numer. Anal. Math. Model.* **34**, 215–223 (2019).
- Zehnder, J., Li, Y., Coros, S. & Thomaszewski, B. NTopo: mesh-free topology optimization using implicit neural representations. In *Proc. Advances in Neural Information Processing Systems* Vol. 34 (eds Ranzato, M. et al.) 10368–10381 (Curran Associates, 2021).
- Bhaffikatti, S. *Finite Element Analysis* (New Age International, 2005).
- Vaxman, A., Ben-Chen, M. & Gotsman, C. A multi-resolution approach to heat kernels on discrete surfaces. *ACM Trans. Graph.* **29**, 121 (2010).
- Sharp, N., Attaki, S., Crane, K. & Ovsjanikov, M. Diffusionnet: Discretization agnostic learning on surfaces. *ACM Trans. Graph.* **41**, 27 (2022).
- Liu, H.-T. D., Kim, V. G., Chaudhuri, S., Aigerman, N. & Jacobson, A. Neural subdivision. *ACM Trans. Graph.* **39**, 124 (2020).
- Qi, C. R., Su, H., Mo, K. & Guibas, L. J. PointNet: deep learning on point sets for 3D classification and segmentation. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition* 652–660 (IEEE, 2017).
- Qi, C. R., Yi, L., Su, H. & Guibas, L. J. PointNet++: deep hierarchical feature learning on point sets in a metric space. *Proc. Advances in Neural Information Processing Systems* Vol. 30 (eds Guyon, I.) (Curran Associates, 2017).
- Spielberg, A., Amini, A., Chin, L., Matusik, W. & Rus, D. Co-learning of task and sensor placement for soft robotics. *IEEE Robot. Autom. Lett.* **6**, 1208–1215 (2021).
- Wang, Y. et al. Dynamic graph CNN for learning on point clouds. *ACM Trans. Graph.* **38**, 146 (2019).
- Park, J. J., Florence, P., Straub, J., Newcombe, R. & Lovegrove, S. DeepSDF: learning continuous signed distance functions for shape representation. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* 165–174 (IEEE, 2019).
- Monti, F. et al. Geometric deep learning on graphs and manifolds using mixture model CNNs. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition* 5115–5124 (IEEE, 2017).
- Chang, A. X. et al. ShapeNet: an information-rich 3D model repository. Preprint at <https://arxiv.org/abs/1512.03012> (2015).
- Yang, G. et al. Pointflow: 3D point cloud generation with continuous normalizing flows. In *Proc. IEEE/CVF International Conference on Computer Vision* 4541–4550 (IEEE, 2019).
- Yang, G., Belongie, S., Hariharan, B. & Koltun, V. Geometry processing with neural fields. In *Proc. Advances in Neural Information Processing Systems* Vol. 34 (eds Ranzato, M. et al.) 22483–22497 (Curran Associates, 2021).
- Wang, Y. & Solomon, J. M. Deep closest point: learning representations for point cloud registration. In *Proc. IEEE/CVF International Conference on Computer Vision* 3523–3532 (IEEE, 2019).
- Teran, J., Sifakis, E., Irving, G. & Fedkiw, R. Robust quasistatic finite elements and flesh simulation. In *Proc. 2005 ACM SIGGRAPH/Eurographics Symposium on Computer Animation* (eds Hansmann, W. et al.) 181–190 (ACM, 2005).
- Bächer, M., Knoop, E. & Schumacher, C. Design and control of soft robots using differentiable simulation. *Curr. Robot. Rep.* **2**, 211–221 (2021).
- Hu, Y. et al. ChainQueen: a real-time differentiable physical simulator for soft robotics. In *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 6265–6271 (IEEE, 2019).
- Du, T. et al. DiffPD: differentiable projective dynamics. *ACM Trans. Graph.* **41**, 13 (2021).

44. Qiao, Y., Liang, J., Koltun, V. & Lin, M. Differentiable simulation of soft multi-body systems. In *Proc. Advances in Neural Information Processing Systems* Vol. 34 (eds Ranzato, M. et al.) 5115–5124 (Curran Associates, 2021).
45. Dubied, M., Michelis, M. Y., Spielberg, A. & Katzschmann, R. K. Sim-to-real for soft robots using differentiable FEM: recipes for meshing, damping and actuation. *IEEE Robot. Autom. Lett.* **7**, 5015–5022 (2022).
46. de Avila Belbute-Peres, F., Smith, K., Allen, K., Tenenbaum, J. & Kolter, J. Z. End-to-end differentiable physics for learning and control. *Adv. Neural Inf. Process. Syst.* **31**, 7178–7189 (2018).
47. Chen, R. T., Rubanova, Y., Bettencourt, J. & Duvenaud, D. K. Neural ordinary differential equations. In *Proc. Advances in Neural Information Processing Systems* Vol. 34 (eds Bengio, S. et al.) 6572–6583 (Curran Associates, 2018).
48. Long, Z., Lu, Y., Ma, X. & Dong, B. PDE-Net: learning PDEs from data. In *Proc. International Conference on Machine Learning* (eds Dy, J. & Krause, A.) 3208–3216 (PMLR, 2018).
49. Sanchez-Gonzalez, A. et al. Learning to simulate complex physics with graph networks. In *Proc. International Conference on Machine Learning* (eds Daumé III, H. & Singh, A.) 8459–8468 (PMLR, 2020).
50. Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A. & Battaglia, P. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations* (2020).
51. Degraeve, J., Hermans, M. & Dambre, J. et al. A differentiable physics engine for deep learning in robotics. *Front. Neurobot.* **13**, 6 (2019).
52. Freeman, C. D. et al. Brax—a differentiable physics engine for large scale rigid body simulation. In *35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)* (2021).
53. Werling, K., Omens, D., Lee, J., Exarchos, I. & Liu, C. K. Fast and feature-complete differentiable physics for articulated rigid bodies with contact. In *Proc. Conference of Robotics: Science and Systems (RSS)* (2021); <https://doi.org/10.15607/RSS.2021.XVII.034>
54. Spielberg, A., Du, T., Hu, Y., Rus, D. & Matusik, W. Advanced soft robot modeling in ChainQueen. *Robotica* **41**, 74–104 (2021).
55. Qiao, Y.-L., Liang, J., Koltun, V. & Lin, M. C. Scalable differentiable physics for learning and control. In *Proc. 37th International Conference on Machine Learning* (eds Daumé III, H. & Singh, A.) 7847–7856 (PMLR, 2020).
56. Raissi, M., Yazdani, A. & Karniadakis, G. E. Hidden fluid mechanics: learning velocity and pressure fields from flow visualizations. *Science* **367**, 1026–1030 (2020).
57. Wiewel, S., Becher, M. & Thuerey, N. Latent space physics: towards learning the temporal evolution of fluid flow. *Comput. Graph. Forum* **38**, 71–82 (2019).
58. Hu, Y. et al. QuanTaichi: a compiler for quantized simulations. *ACM Trans. Graph.* **40**, 182 (2021).
59. Li, W. & Todorov, E. Iterative linear quadratic regulator design for nonlinear biological movement systems. In *Proc. 1st International Conference on Informatics in Control, Automation and Robotics* 222–229 (IEEE, 2004).
60. Posa, M., Cantu, C. & Tedrake, R. A direct method for trajectory optimization of rigid bodies through contact. *Int. J. Robot. Res.* **33**, 69–81 (2014).
61. Gifftthaler, M., Neunert, M., Stäuble, M. & Buchli, J. The control toolbox—an open-source C++ library for robotics, optimal and model predictive control. In *Proc. 2018 IEEE International Conference on Simulation, Modeling and Programming for Autonomous Robots (SIMPAR)* (eds Kurniawati, H. et al.) 123–129 (IEEE, 2018).
62. Tedrake, R. & the Drake Development Team. Drake: a planning, control and analysis toolbox for nonlinear dynamical systems (MIT, 2016); <http://drake.mit.edu>
63. Agrawal, A. et al. Differentiable convex optimization layers. In *Proc. Advances in Neural Information Processing Systems* Vol. 32 (eds Wallach, H. et al.) 9562–9574 (Curran Associates, 2019).
64. Amos, B., Jimenez, I., Sacks, J., Boots, B. & Kolter, J. Z. Differentiable MPC for end-to-end planning and control. In *Proc. 32nd International Conference on Neural Information Processing Systems* Vol. 34 (eds Bengio, S. et al.) 8299–8310 (Curran Associates, 2018).
65. Pfrommer, S., Halm, M. & Posa, M. ContactNets: learning discontinuous contact dynamics with smooth, implicit representations. In *Proc. Conference on Robot Learning* (eds Faust, A. et al.) 2279–2291 (PMLR, 2021).
66. Xu, J. et al. Accelerated policy learning with parallel differentiable simulation. In *Proc. International Conference on Learning Representations* (2022).
67. Ajay, A. et al. Combining physical simulators and object-based networks for control. In *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 3217–3223 (IEEE, 2019).
68. Hwangbo, J. et al. Learning agile and dynamic motor skills for legged robots. *Sci. Robot.* **4**, eaau5872 (2019).
69. Ilyas, A. et al. A closer look at deep policy gradients. In *International Conference on Learning Representations* (2020).
70. Metz, L., Freeman, C. D., Schoenholz, S. S. & Kachman, T. Gradients are not all you need. Preprint at <https://arxiv.org/abs/2111.05803> (2021).
71. Suh, H. J., Simchowitz, M., Zhang, K. & Tedrake, R. Do differentiable simulators give better policy gradients? In *Proc. International Conference on Machine Learning* 20668–20696 (PMLR, 2022).
72. Pharr, M., Jakob, W. & Humphreys, G. *Physically Based Rendering: from Theory to Implementation* (Morgan Kaufman, 2016).
73. Dudchik, Y. I. & Kolchevsky, N. A microcapillary lens for X-rays. *Nucl. Instrum. Methods Phys. Res. A* **421**, 361–364 (1999).
74. Li, T.-M., Aittala, M., Durand, F. & Lehtinen, J. Differentiable Monte Carlo ray tracing through edge sampling. *ACM Trans. Graph.* **37**, 222 (2018).
75. Loubet, G., Holzschuch, N. & Jakob, W. Reparameterizing discontinuous integrands for differentiable rendering. *ACM Trans. Graph.* **38**, 228 (2019).
76. Zhang, C., Miller, B., Yan, K., Gkioulekas, I. & Zhao, S. Path-space differentiable rendering. *ACM Trans. Graph.* **39**, 143 (2020).
77. Bangaru, S. P. et al. Systematically differentiating parametric discontinuities. *ACM Trans. Graph.* **40**, 107 (2021).
78. Kato, H., Ushiku, Y. & Harada, T. Neural 3D mesh renderer. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition* 3907–3916 (IEEE, 2018).
79. Hu, Y., Li, T.-M., Anderson, L., Ragan-Kelley, J. & Durand, F. Taichi: a language for high-performance computation on spatially sparse data structures. *ACM Trans. Graph.* **38**, 201 (2019).
80. Hu, Y. et al. DiffTaichi: differentiable programming for physical simulation. In *International Conference on Learning Representations* (2020).
81. Johnson, J. et al. Accelerating 3D deep learning with PyTorch3D. *ACM Digital Library* <https://doi.org/10.1145/3415263.3419160> (2020).
82. Bailey, P. et al. Differentiable graphics with TensorFlow 2.0. In *ACM SIGGRAPH 2019 Courses* 1–211 (ACM, 2019).
83. Jakob, W., Speierer, S., Roussel, N. & Vicini, D. Dr.JIT: a just-in-time compiler for differentiable rendering. *ACM Trans. Graph.* **41**, 124 (2022).
84. Tewari, A. et al. Advances in neural rendering. *Comput. Graph. Forum* **41**, 703–735 (W2022).
85. Gkioulekas, I., Zhao, S., Bala, K., Zickler, T. & Levin, A. Inverse volume rendering with material dictionaries. *ACM Trans. Graph.* **32**, 162 (2013).

86. LeGendre, C. et al. DeepLight: learning illumination for unconstrained mobile mixed reality. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* 5918–5928 (IEEE, 2019).
87. Weiss, S. & Westermann, R. Differentiable direct volume rendering. *IEEE Trans. Vis. Comput. Graph.* **28**, 562–572 (2021).
88. Vicini, D., Speierer, S. & Jakob, W. Path replay backpropagation: differentiating light paths using constant memory and linear time. *ACM Trans. Graph.* **40**, 108 (2021).
89. Shi, L. et al. Match: Differentiable material graphs for procedural material capture. *ACM Trans. Graph.* **39**, 196 (2020).
90. Habermann, M., Xu, W., Zollhofer, M., Pons-Moll, G. & Theobalt, C. DeepCap: monocular human performance capture using weak supervision. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* 5052–5063 (IEEE, 2020).
91. Ritchie, D., Thomas, A., Hanrahan, P. & Goodman, N. Neurally-guided procedural models: amortized inference for procedural graphics programs using neural networks. In *Proc. Advances in Neural Information Processing Systems* Vol. 29 (eds Lee, D. et al.) (Curran Associates, 2016).
92. AlQuraishi, M. & Sorger, P. K. Differentiable biology: using deep learning for biophysics-based and data-driven modeling of molecular mechanisms. *Nat. Methods* **18**, 1169–1180 (2021).
93. Schoenholz, S. & Cubuk, E. D. JAX, MD: a framework for differentiable physics. *Proc. Advances in Neural Information Processing Systems* Vol. 33 (eds Larochelle, H. et al.) (Curran Associates, 2020).
94. Innes, M. et al. A differentiable programming system to bridge machine learning and scientific computing. Preprint at <https://arxiv.org/abs/1907.07587> (2019).
95. Moses, W. S. et al. Reverse-mode automatic differentiation and optimization of GPU kernels via enzyme. In *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis* 1–16 (ACM, 2021).
96. Bettencourt, J., Johnson, M. J. & Duvenaud, D. Taylor-mode automatic differentiation for higher-order derivatives in JAX. Preprint at <https://openreview.net/forum?id=SkxEF3FNPH> (2019).
97. Ragan-Kelley, J. et al. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *ACM SIGPLAN Notices* **48**, 519–530 (2013).
98. Mullapudi, R. T., Adams, A., Sharlet, D., Ragan-Kelley, J. & Fatahalian, K. Automatically scheduling halide image processing pipelines. *ACM Trans. Graph.* **35**, 1–11 (2016).
99. Griewank, A. & Walther, A. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation* (Society for Industrial and Applied Mathematics, 2008).

Acknowledgements

We thank A. Mordvintsev, A. Tagliasacchi, J. Valentin, O. Sanseviero and S. Bouaziz for early discussions on content and structure. This work was supported by a UKRI Future Leaders Fellowship (grant number G104084).

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Andrew Spielberg or Derek Nowrouzezahrai.

Peer review information *Nature Machine Intelligence* thanks Jinwei Ye, Thomas Mueller, and the other, anonymous, reviewer for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© Springer Nature Limited 2023